Learn Web Development: Start Here

Fulllion Creative Works

Contents

1. How Web Apps Work
2. Front End, Back End or Full Stack
3. The Five Recommended Steps
4. Free Curricula Compared
5. More Course and Practice Sites
6. Your Beginner's Toolkit
7. Learning Without Burning Out
8. Good First Videos
9. Where to Go Next
10. FAQ
11. Glossary
12. Sources

---

1. How Web Apps Work

Every website and web app, from a one-page portfolio to an online store, is built from the same few pieces. Knowing them first makes everything else easier to place.

> **Watch next:** Once the table below makes sense, Everything You NEED to Know About WEB APP Architecture by **ForrestKnight** (June 2022) shows how those pieces get arranged. It covers:
>
> - **client-server**, which almost every site uses, versus **peer-to-peer**;
> - **tiers:** whether the front end, back end and database share one machine or each get their own;
> - **monolith, microservices and serverless:** one codebase, many small services, or functions run by a cloud provider, using YouTube and an online shop as examples.
>
> It's more advanced than this section, so don't worry if it doesn't all stick yet. The video presents monoliths as mostly a drawback. In practice, many experienced developers suggest starting with a monolith and splitting it up only when you need to (Martin Fowler's MonolithFirst), and that's the right choice for your own projects.

Piece	What it is	Everyday example
Client	The program the user sees, usually a web browser such as Chrome, Firefox, Edge or Safari.	Your browser showing an online shop.

Piece	What it is	Everyday example
Server	A computer that waits for requests and sends back pages or data.	The shop's computer that receives "show me the shoes".
Front end	Everything that runs in the browser: the page's structure (HTML), its look (CSS) and its behavior (JavaScript).	The product grid, the menu, the **Add to cart** button.
Back end	The code on the server: it checks who you are, applies the rules and talks to the database.	Checking that the shoes are in stock, and working out the price.
Database	Where data is stored so it survives after the page closes.	The product list, user accounts and orders.
API	A defined way for one program to ask another for data or actions, usually over HTTP, often returning JSON.	The front end asking `/api/products` for the shoe list.
Hosting	Renting space on a server (or a cloud service) so your site is online all the time.	GitHub Pages, Firebase, or any web host.
Domain and DNS	A domain is the name people type. DNS is the internet's address book, turning that name into the server's address.	`example.com` pointing to the shop's server.

What happens when you open a page

1. You type an address, or click a link.
2. DNS turns the domain name into the server's address.
3. Your browser sends an **HTTP request** to that server (over HTTPS, so it's encrypted).
4. The server's back end runs, fetches anything it needs from the database, and sends back a **response**: an HTML page, or data such as JSON.
5. Your browser reads the HTML, downloads the CSS, JavaScript and images it links to, and draws the page.
6. JavaScript in the page can keep sending requests to APIs without reloading, which is how feeds, chats and shopping carts update in place.

MDN's How the web works covers this in more depth, and How browsers load websites covers step 5.

> **A simple site doesn't need a back end.** A portfolio or a restaurant's menu page can be plain HTML, CSS and JavaScript files on a static host. You add a back end and a database when the site needs to remember things, such as accounts, posts or orders.

---

2. Front End, Back End or Full Stack

Path	You build	Core skills	Choose it if you
Front end	What people see and click	HTML, CSS, JavaScript, then a framework such as React	Like visual results, design and how things feel to use
Back end	The server, the data and the rules	A server language (Python, JavaScript with Node.js, Java and others), SQL, APIs, security	Like logic, data and solving puzzles out of sight

Path	You build	Core skills	Choose it if you
Full stack	Both, and how they connect	Front end and back end, plus deployment	Want to build whole apps on your own, or aren't sure yet

How to choose:

1. **Not sure? Start with the front end.** HTML, CSS and JavaScript are needed on every path, and you see results straight away. That helps you stay motivated.
2. **Try a small project in each.** Build a page, then a tiny API, and notice which you enjoy more.
3. **Look at job ads near you,** or for the remote roles you want, and note which skills keep appearing.
4. **You can change your mind.** Skills carry over. Plenty of back end developers started on the front end, and full stack developers usually started on one side.

Each path has its own guide, and each one applies the five steps below to its own skills:

- Learn Front End Web Development, Section 2: The Five Steps
- Learn Back End Web Development, Section 3: The Minimum to Learn First
- Full Stack Web Development and Stacks, Section 2: The Route In

---

3. The Five Recommended Steps

These five steps work for every path. **You can learn the topics within each step in any order, but don't take on too many at once.** One language or tool at a time beats five at half speed.

Step 1: Learn a language to a basic proficiency

Basic proficiency means you can write small programs without copying them line by line. You know the core ideas: variables, conditions (`if`), loops, functions, and lists or arrays.

Path	Start with	You're there when you can
Front end	HTML and CSS, then JavaScript	Build a page with headings, links, images and a form; lay it out with Flexbox; make a button show or hide something
Back end	Python (or JavaScript)	Write a script that reads a file, loops over its lines, and prints a summary
Either	SQL	Write a query that picks rows from a table, filters them, and sorts them

Step 2: Build basic applications and projects

Build small things **without following a tutorial step by step.** This is where learning sticks. Some classic first projects, taken from The Odin Project's Foundations course:

- A **recipe page**: plain HTML pages linked together.
- A **landing page**: a styled single page laid out with Flexbox.
- **Rock Paper Scissors**: JavaScript logic in the browser console, then with buttons.
- **Etch-a-Sketch**: a grid of squares that change color as you move the mouse.

- A **calculator**: buttons, a display, and the logic behind them.

For a back end start, try a command-line to-do list in Python that saves to a file.

Step 3: Learn libraries, frameworks and runtime environments

Once the basics feel comfortable, learn the tools built on top of your language. They save you from writing the same things over and over.

- **Library:** code you call when you need it, such as React for building user interfaces.
- **Framework:** a structure you build inside, which calls your code, such as Next.js, Django or Express.
- **Runtime environment:** the program that runs your code outside the browser, such as Node.js for JavaScript.

Examples by path:

- **Front end:** React (with Next.js later), or Vue. For styling, a CSS framework such as Tailwind CSS or Bootstrap, once you're comfortable with plain CSS.
- **Back end:** Node.js with Express, or Python with Flask or Django.

> **Learn the language before the framework.** A framework makes much more sense when you know what it's doing for you. To see what a framework does for you, watch What is a Framework and Why use Frameworks by Bitfumes (now Sarthaksavvy, 2017). He describes a framework as a ready-made skeleton, like the frame of a house. It gives you:
>
> - a standard way to structure an app;
> - reusable functions, such as fetching data from the database;
> - routing, which decides how each URL is answered;
> - the model-view-controller (MVC) pattern;
> - easy use of third-party packages.
>
> His examples are PHP frameworks such as Laravel, but the ideas apply to Django, Express and the rest.

Step 4: Build bigger projects, with Git and GitHub

Now combine pieces: a framework, more than one language, and real data. **Use Git and GitHub from here on,** if you haven't already, and give every project its own repository.

Example projects:

- A **weather app** that calls a public weather API and shows the forecast.
- A **to-do list** that saves to the browser's storage, then later to a database.
- A **message board** or **blog** with sign-in, a database, and pages to create, edit and delete posts.
- A **small online shop** with a product list, a cart, and an admin page.

Good habits for this step:

1. **Commit small and often,** with messages that say what changed.
2. **Write a README** for each project: what it does, how to run it, and a screenshot.
3. **Put it online.** A live link is far more convincing than a folder of code. Section 6 and the careers guide cover where.

Step 5: Create a portfolio website and pursue your career

Make a simple site that shows your best three to five projects, with live links, code links and a short write-up of each. Then start applying, networking or freelancing. The Web Developer Careers guide covers portfolios, GitHub profiles, applications and freelancing in detail.

---

4. Free Curricula Compared

A curriculum gives you an order to learn things in, so you're not guessing. **Pick one as your main path.** Use the others as references.

Curriculum	Cost	What it is	Certificate	Best for
The Odin Project	Free, funded by donations	Open source. Foundations, then **Full Stack JavaScript** or **Full Stack Ruby on Rails**. Lessons point to the best articles and docs, and you build dozens of projects locally on your own computer. Active Discord community.	None	Self-starters who want real-world setup and lots of projects
freeCodeCamp	Free, a charity	Interactive lessons in the browser. Its new curriculum has six certifications (Responsive Web Design, JavaScript, Front End Libraries, Python, Relational Databases, and Back End Development and APIs), each with projects and an exam, leading to a **Certified Full Stack Developer** capstone.	Free, verified	Structured, hands-on practice with free certificates
MDN Learn web development	Free	Written lessons from the makers of Firefox, in Getting started, Core and Extension modules. Aims to take you from beginner to comfortable.	None	Clear explanations, and the most trusted reference
CS50's Web Programming with Python and JavaScript (CS50W)	Free to take; optional paid edX certificate	Harvard's course: HTML and CSS, Git, Python, Django, SQL, JavaScript, user interfaces, testing and CI/CD, and scalability and security, plus a final project. Expects CS50 or some prior programming.	Free CS50 certificate, or a paid edX one	Learners with some coding who want a university-style course
roadmap.sh	Free to browse	Not a course but a map: community-made role roadmaps (Frontend, Backend, Full Stack) showing what to learn and in what order, with project ideas.	None	Seeing the whole picture, and checking for gaps

Details worth knowing:

- **freeCodeCamp's older certifications** are now called "legacy". In September 2025, freeCodeCamp said they will eventually expire, no sooner than 2028, and advised learners to move to the new curriculum. It estimated about 300 hours per certification, and about 1,800 hours for the whole full stack path. Some parts were released in stages, so check the Learn page for what's live.
- **CS50W** is free through Harvard's OpenCourseWare at cs50.harvard.edu/web. Score at least 70% on every project, including the final project, and you can get a free CS50 certificate. If you want a verified certificate from edX instead, enroll through edX. As of November 2026, edX lists it as free to audit, self-paced, about 12 weeks at 6 to 9 hours a week, with a verified certificate for US$299, and the current run open until June 30, 2027.
- **HarvardX's Computer Science for Web Programming Professional Certificate** pairs CS50's Introduction to Computer Science (CS50x) with CS50W. It's still offered on edX, as a paid bundle of the two certificates.
- **roadmap.sh's roadmaps** are kept up to date by the community, and its project pages let you submit a solution and see other people's.

---

5. More Course and Practice Sites

These are good extras alongside your main curriculum, especially when one explanation doesn't click and you need another.

Courses

Site	Cost	Good for
Codecademy	Free Basic plan. Paid plans add portfolio projects and certificates, and the Pro plan adds career paths.	Short interactive lessons in the browser
Khan Academy computer programming	Free	Gentle starts, including Intro to JS, Intro to HTML/CSS and Intro to SQL
web.dev Learn	Free	Courses written with members of Google's Chrome team, such as Learn HTML, Learn CSS, Learn JavaScript, Learn Accessibility and Learn Performance
edX	Many courses free to audit. Certificates are paid, and financial assistance is offered on many.	University courses from Harvard, MIT and others
Class Central	Free to search	Finding free university courses on any topic
freeCodeCamp's list of 860+ free CS courses	Free	A big list from universities, compiled by Class Central's founder (January 2023, so some links may have moved)
Educative	Mostly paid, with a **Free** filter	Text-based courses with in-browser coding

References

- **MDN Web Docs** is the reference to trust for HTML, CSS and JavaScript. Bookmark it.
- **W3Schools** has free, short tutorials and a "Try it" editor, which many beginners find friendly. Its certificates are paid. When the two disagree, go with MDN, which is more thorough and more closely tied to the official web standards.

Light practice on your phone

Sololearn and Mimo offer bite-sized lessons on the web, iOS and Android, with free tiers and paid plans. They're good for keeping ideas fresh on a commute, but treat them as an **accessory** to a real curriculum, because you won't build full projects in them.

Coding challenges and project sites

Challenge sites (Codewars, HackerRank, Frontend Mentor and others) are covered in Coding Practice and Interview Prep. Project ideas for your portfolio are in the careers guide.

---

6. Your Beginner's Toolkit

You need four things, and all of them are free.

6.1 A code editor

Editor	Cost	Notes
Visual Studio Code	Free	The most used editor by far: 75.9% of respondents in Stack Overflow's 2025 survey. Huge extension library. Windows, macOS and Linux. **Start here if unsure.**
Neovim	Free, open source	A "hyperextensible Vim-based text editor" that runs in the terminal. Powerful and fast, but it has a steep learning curve. Try it later.
Kate	Free, from KDE	A capable editor for Linux, Windows and macOS.
Sublime Text	Free to evaluate; a license is required for continued use	Very fast. It reminds you to buy a license.

> **VS Code isn't Visual Studio.** Visual Studio is Microsoft's separate, heavier IDE, used mostly for C# and C++. For web development, download **Visual Studio Code** from code.visualstudio.com.
>
> **Skip Atom.** It was a popular free editor, but GitHub retired it on December 15, 2022, and it no longer gets updates.

To install VS Code (and Git, Node.js and Python) with one command each, see the developer setup section of the CLI Cheat Sheet.

VS Code tips worth a minute each. Three come from the official **Visual Studio Code** channel, and one from **Web Dev Simplified**:

Short	What it shows
Balance Out	Run **Emmet: Balance (outward)** from the Command Palette (`Ctrl` + `Shift` + `P`) to select a whole HTML element, closing tag included. Run it again to grow the selection. **Balance (inward)** shrinks it. The presenter binds them to his own keyboard shortcuts in **Keyboard Shortcuts** settings.
To the left, to the left	**Go Back** jumps to where your cursor was before, for example after scrolling up to check a function: `Alt` + `Left` on Windows, `Ctrl` + `-` on macOS, `Ctrl` + `Alt` + `-` on Linux.
Terminal editors	Drag a terminal tab into the editor area, so terminals sit beside your files and show up in **Quick Open** (`Ctrl` + `P`).
5 Must Know VSCode Shortcuts	`Ctrl` + `B` hides the sidebar. `Ctrl` + `Shift` + `E` and `Ctrl` + `Shift` + `F` jump to the file explorer and search. `Ctrl+`` toggles the terminal. `Ctrl` +click goes to where something is defined. `Ctrl` + `.` offers quick fixes. `Ctrl` + `Space` shows suggestions. On a Mac, use `Cmd` for most of these.

6.2 A browser with developer tools

Every major browser has built-in **developer tools (DevTools)**. They let you inspect and change a page's HTML and CSS live, see JavaScript errors, and watch every network request. You'll use them every day.

Browser	Open DevTools	Useful for
Chrome (and Edge)	`F12` or `Ctrl` + `Shift` + `I`. On a Mac, `Cmd` + `Option` + `I`.	The **Elements**, **Console**, **Network** and **Performance** panels
Firefox	`F12` or `Ctrl` + `Shift` + `I`. On a Mac, `Cmd` + `Option` + `I`.	The **Page Inspector** (great for Grid and Flexbox), **Responsive Design Mode** and the **Accessibility** inspector

Try it now: right-click any part of this page and choose **Inspect**.

6.3 Git and GitHub

- **Git** is version control: it saves snapshots of your project, so you can see what changed and undo mistakes. It runs on your computer.
- **GitHub** is a website that stores your Git projects online, shows them to employers, and lets you work with others. Make an account early and keep your work there.

After installing Git (see the CLI Cheat Sheet), tell it who you are, once:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Use the email address on your GitHub account, or GitHub's private `noreply` address from your GitHub email settings if you'd rather not publish your real one.

Check your tools are installed:

```
git --version
code --version
```

Then:

- GitHub's Hello World tutorial teaches repositories, branches and pull requests in the browser.

- Publishing to GitHub with gh shows how to put a project on GitHub from the terminal.
- The CLI Cheat Sheet has everyday Git commands side by side.

6.4 Online editors, for quick experiments

You don't need to install anything to try an idea. These run in the browser:

Site	Good for
CodePen	Front end experiments in HTML, CSS and JavaScript, and browsing other people's demos. A paid tier adds private pens.
StackBlitz	Full Node.js projects running inside your browser, such as React or Vite starters.
CodeSandbox	Front end and full stack sandboxes. Its free plan has usage limits.
Replit	Many languages in the browser. It now centers on its AI app-building agent, and its free use is limited.

Use them for experiments and sharing, but build your real projects locally with VS Code and Git. That's how you'll work in a job.

---

7. Learning Without Burning Out

Most people who stop learning to code don't fail at the hard parts. They run out of steam. These habits help.

One language at a time

Get comfortable with one language before adding another. Learning JavaScript, Python and SQL all at once means you're a beginner in three things at the same time. The exception is HTML and CSS, which go together.

Build more than you watch

Tutorials feel productive, but following along isn't the same as being able to do it. Getting stuck in a loop of one tutorial after another is often called **tutorial hell**. To get out:

1. After each tutorial, **build something similar without it.** Change the topic: if the tutorial built a to-do list, build a reading list.
2. **Add one feature** the tutorial didn't cover.
3. When stuck, search for the specific problem, not a new tutorial.

Learning How to Learn Programming, Section 4 has more ways out.

Spaced practice

Short, regular sessions beat long, rare ones. Your memory holds on to things better when you come back to them over several days.

- **Code a little most days,** for example 45 to 90 minutes, rather than one exhausting weekend session.

- **Revisit older topics.** Rebuild a small earlier project from memory every few weeks.
- **Test yourself** before looking things up: try to write it first, then check.
- **Take real breaks.** MDN's learning advice suggests a proper 15-minute break every hour to 90 minutes, and looking away from the screen regularly.

Look after the basics

Sleep, food and exercise affect how well you learn far more than any app. Set a learning schedule you can keep, schedule time off as well, and reward yourself when you finish something.

Learning How to Learn Programming goes much deeper: study techniques, taking notes, planning projects, staying focused, and books.

---

8. Good First Videos

- Learn web development as an absolute beginner, **Coder Coder** (November 2019). A roadmap in four steps:
  1. The concepts: client and server, front end and back end, and a code editor.
  2. HTML, CSS and JavaScript.
  3. The everyday tools: a package manager (npm), a build tool, and Git.
  4. Then either more front end (Sass, responsive design, and a framework such as React, Angular or Vue) or the back end (a server, a server language, and SQL or NoSQL databases).

  She advises learning a little of both sides, and choosing a framework by what local job ads ask for. **Since then:** the Atom editor it mentions was retired in 2022 (Section 6.1). Its build tools (Gulp, webpack and Parcel) have largely given way to newer ones. React's own docs now suggest starting with Vite, Parcel or Rsbuild.
- Learn Web Development FREE and Actually Get Hired, **devsmak** (the channel is now **James Cross**, May 2022). He left a construction job and taught himself, and he maps a free, front-end-first route:
  - freeCodeCamp's courses first, including the responsive design, JavaScript and front end libraries certifications. He'd skip jQuery, since React apps don't use it.
  - Then The Odin Project's Foundations and Full Stack JavaScript path.
  - Then CS50x for computer science basics.
  - Skip pricey bootcamps.
  - Treat certificates as nice to have. What gets you hired is building your own projects without a tutorial.

  Since then: freeCodeCamp has replaced those certifications with a new curriculum, and the old ones are now "legacy" (Section 4).
- How To Think Like A Programmer, a talk by computer science teacher **Andy Harris**, uploaded by **Coding Tech** (October 2018, about an hour). His argument: programming is mostly planning, not memorizing a language. His method:

    - Write the steps in plain language first, on paper if you like.
    - Turn each step into a code comment, then translate one comment at a time into code, searching for syntax only once you know what you need.
    - When it breaks, ask whether the plan was wrong or the code was. Check your assumptions by printing values and their types, instead of guessing at fixes.

    His examples are in Python. Jump to:

    - the myths about learning to code (9:07)
    - comments first, then code (18:44)
    - a worked example, with a bug (30:42)
    - debugging (37:17)
    - loops (48:07)

    His aside that Python has no switch statement is out of date: Python 3.10 (2021) added `match`.
- Everything You NEED to Know About WEB APP Architecture, **ForrestKnight**. How the pieces fit together, once the basics make sense (Section 1). It's hard on monoliths, but for your own projects, starting with one is usually the better choice (see the note in Section 1).
- What is a Framework and Why use Frameworks, **Bitfumes** (now **Sarthaksavvy**). For Step 3.
- CS50 Lectures 2021, **CS50**. Harvard's full lecture series for CS50x and CS50W, covering C, Python, SQL, HTML, CSS and JavaScript. Newer lectures, such as CS50x 2026, are on cs50.harvard.edu/x. This is one option for a deeper foundation. Computer Science Fundamentals covers it in more detail.

---

9. Where to Go Next

Web development is one path among several. The five steps and the study habits in this guide carry over to all of them, so nothing you learn here is wasted if you change direction.

9.1 The web development path

Guide	Read it when
Learn Front End Web Development	You want to build what people see and click.
Learn Back End Web Development	You want to build servers, APIs and databases.
Full Stack Web Development and Stacks	You want to build whole apps, or are choosing a stack.
Web Developer Careers, Portfolio and Freelancing	You have a few projects and want a job or clients.

9.2 Learning and practice, for any path

Guide	Read it when
Learning How to Learn Programming	You want study techniques that stick, or you're stuck in tutorial hell.

Guide	Read it when
Computer Science Fundamentals	You can build small apps and want data structures, algorithms and Big O. Section 1.2 says what can wait.
Coding Practice and Interview Prep	You want challenge sites, or have a technical interview coming up.
Open Source and Developer Communities	You want help from other developers, your first open source contribution, or to use AI assistants well.

9.3 Other paths

If something else catches your interest, these guides start from zero, just like this one:

Guide	What you'll do	It might suit you if you
Learn Game Development	Make small games with an engine such as Godot or Unity, starting from free courses	Like games, and want to see your code move and react
Learn Data Science	Answer questions with data, using Python, SQL and statistics	Enjoy the SQL and data parts of back end work, or like finding patterns
Getting Started with DevOps	Automate how software is built, tested, shipped and run, with Linux, Git, Docker and the cloud (the order to learn in)	Enjoy deploying your projects more than building their pages
IT Support Careers and Certifications	Help people and organizations keep their computers running, on a path from help desk to networking, security or cloud	Like fixing things and helping people, and want a route into tech that isn't mainly coding
Learn 3D Modeling	Build, texture and animate models in Blender, which is free, starting with your first hour	Like the visual side, and want to make assets for games, animation, 3D on the web, or printing
3D Printing: Getting Started	Print, tune and fix real objects, with a ladder of first projects	Want to make physical things, such as replacement parts, gadgets and props

9.4 Tool guides

Short, practical guides to the tools you'll meet along the way:

- CLI Cheat Sheet - Terminal commands, Git, and installing Node, Python, Git and VS Code with winget.
- Python and Node Tool Environments - Installing Python and Node.js, and giving each project its own environment.
- Safer Package Installs with npm and pip - Avoiding malicious packages when you install libraries.
- Publishing to GitHub with gh - Putting a project on GitHub from the terminal, and keeping secrets out of it.
- Packaging Windows Apps with PyInstaller and Inno Setup - Turning a Python app into a `.exe` and an installer.
- Batch Image and 3D Tools: GIMP, Blender and Pillow - Resizing and converting whole folders of images and models with scripts.
- PDF and Document Tools - Reading, searching and making PDFs and Word documents from code.

- Transcribing Videos with yt-dlp and faster-whisper - Turning videos into searchable text on your own PC.

---

10. FAQ

Do I need a degree? Not always. The US Bureau of Labor Statistics lists a bachelor's degree as typical, but says education requirements for web developers "range from a high school diploma to a bachelor's degree". It also notes that developers may not need specific credentials if they can show their abilities through work or projects. That's what the portfolio in Step 5 is for.

Do I need to be good at math? For most web development, no. Basic arithmetic and clear logical thinking go a long way. Some areas, such as graphics, data work and algorithms, use more. Computer Science Fundamentals covers the math worth knowing.

Do I need to learn computer science first? No. You can build and deploy real web projects first, and pick up data structures, algorithms and Big O as you need them. Computer Science Fundamentals, Section 1.2 suggests what to learn when.

Which language should I learn first? HTML and CSS, then JavaScript. Every web path uses them, and they show results right away. For a back end focus, Python is a friendly next language.

How long will it take? It varies a lot with your time and background. For scale, freeCodeCamp estimates about 1,800 hours for its full stack curriculum. Steady progress over months matters more than speed.

Should I use AI coding assistants? Most developers do: 84% of respondents to Stack Overflow's 2025 survey use AI tools or plan to. But 46% said they distrust the accuracy of AI output, compared with 33% who trust it. While you're learning, write code yourself first, so you understand it well enough to spot when an AI suggestion is wrong. Never paste passwords, keys or private data into one. Open Source and Developer Communities covers using them responsibly.

Where do I ask for help? Your curriculum's community first (The Odin Project and freeCodeCamp both have one), then the communities listed in Open Source and Developer Communities. Say what you tried, what you expected, and what happened, and include the error message.

---

11. Glossary

Term	Meaning
API	A defined way for programs to talk to each other. Web APIs usually take requests over HTTP and answer with JSON.
Accessibility	Building sites everyone can use, including people using screen readers, keyboards only, or zoom.
Back end	Code that runs on the server: logic, data and security.
Browser	The app that loads and shows web pages, such as Chrome, Firefox, Edge or Safari.
CSS	The language that styles a page: colors, fonts, spacing and layout.

Term	Meaning
Database	Organized, stored data an app can query and update.
Deploy	Put a version of your site or app online.
DevTools	The developer tools built into browsers, for inspecting and debugging pages.
DNS	The system that turns domain names into server addresses.
Domain	A website's name, such as `example.com`.
Framework	A ready-made structure you build your app inside, such as Next.js, Django or Express.
Front end	Code that runs in the browser: HTML, CSS and JavaScript.
Full stack	Working on both the front end and the back end.
Git	Version control software that tracks changes to your code.
GitHub	A website that hosts Git repositories, for sharing and collaborating.
Hosting	A server or service that keeps your site online.
HTML	The language that structures a page's content: headings, paragraphs, links, images and forms.
HTTP and HTTPS	The rules browsers and servers use to send requests and responses. HTTPS is the encrypted version.
IDE	Integrated development environment: an editor with built-in tools for running and debugging code.
JavaScript	The programming language of the browser. With Node.js, it also runs on servers.
JSON	A plain-text format for sending data, which looks like `{"name": "Ada"}`.
Library	Reusable code you call when you need it, such as React.
Repository (repo)	A project folder tracked by Git, including its full history.
Responsive design	Layouts that adapt to any screen size, from phones to wide monitors.
REST	A common style for designing web APIs around URLs and HTTP methods such as GET and POST.
Runtime	The program that runs your code, such as Node.js for JavaScript outside the browser.
Server	A computer or program that answers requests from clients.
SQL	The language for querying relational databases.
Static site	A site made of fixed files, with no back end code running on each request.

---

12. Sources

Video credited

- ForrestKnight, Everything You NEED to Know About WEB APP Architecture (June 2022)
- Coder Coder, Learn web development as an absolute beginner (November 2019)
- devsmak (James Cross), Learn Web Development FREE and Actually Get Hired (May 2022)

- Andy Harris, How To Think Like A Programmer, uploaded by Coding Tech (October 2018)
- Bitfumes (Sarthaksavvy), What is a Framework and Why use Frameworks (September 2017)
- CS50, CS50 Lectures 2021
- Visual Studio Code, Balance Out, To the left, to the left and Terminal editors
- Web Dev Simplified, 5 Must Know VSCode Shortcuts

How the web works

- MDN, How the web works
- MDN, How browsers load websites
- Martin Fowler, MonolithFirst (June 2015)
- MDN glossary: API, REST, JSON, HTTP, DNS, SQL

Curricula

- The Odin Project, home, paths and Foundations course
- freeCodeCamp, Learn, Introducing freeCodeCamp checkpoint certifications (September 2025)
- MDN, Learn web development and Research and learning
- CS50, CS50W, CS50W certificate and CS50x
- edX, CS50's Web Programming with Python and JavaScript and Computer Science for Web Programming Professional Certificate
- roadmap.sh, roadmaps and projects

Other courses and references

- Codecademy pricing, Khan Academy, web.dev Learn, W3Schools, Class Central, Educative, Sololearn, Mimo
- freeCodeCamp, 860+ Free Online Programming & Computer Science Courses (January 2023)

Tools

- Visual Studio Code and its Code Navigation docs (Go Back shortcuts), Neovim, Kate, Sublime Text licensing
- GitHub blog, Sunsetting Atom
- Chrome for Developers, Open Chrome DevTools
- Firefox, DevTools user docs and Responsive Design Mode
- Git downloads, GitHub Docs Hello World
- React, Build a React app from Scratch (Vite, Parcel and Rsbuild)
- CodePen, StackBlitz, CodeSandbox, Replit pricing

Facts and figures

- Stack Overflow, 2025 Developer Survey (AI tools and trust) and Technology (editors)
- US Bureau of Labor Statistics, Web Developers and Digital Designers

Latest version: github.com/ImmuneMoon/Learn-Web-Development-Start-Here

If this helped, you can support Fulllion Creative Works through PayPal or Ko-fi.

© 2026 Fulllion Creative Works. All rights reserved.